

BÁO CÁO BÀI TẬP LẬP TRÌNH MẠNG

Họ tên: Hoàng Tuấn Minh

MSSV: 20235376

Mã lớp: 168503

Link bài tập: [hoangtminh/Lap-trinh-mang](https://github.com/hoangtminh/Lap-trinh-mang)

1. Bài tập 3-02

Sử dụng kỹ thuật đa luồng, lập trình ứng dụng chat nhóm 2 người như sau:

- Mỗi client khi kết nối đến server sẽ được lưu vào hàng đợi, nếu số lượng client trong hàng đợi là 2 thì 2 client này sẽ được ghép cặp với nhau.
- Khi 2 client đã được ghép cặp thì tin nhắn gửi từ client này sẽ được chuyển tiếp sang client kia và ngược lại.
- Nếu 1 trong 2 client ngắt kết nối thì client còn lại cũng được ngắt kết nối.

```
multithread > C bai3-02.c > chat_pair_thread(void *)
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <sys/types.h>
6  #include <arpa/inet.h>
7  #include <unistd.h>
8  #include <pthread.h>
9  #include <sys/select.h>
10
11 int waiting_client = -1;
12 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
13
14 struct ClientPair {
15     int client1;
16     int client2;
17 };
18
19 void *chat_pair_thread(void *param) {
20     struct ClientPair *pair = (struct ClientPair *)param;
21     int c1 = pair->client1;
22     int c2 = pair->client2;
23     free(pair);
24
25     char buf1[256], buf2[256];
26     fd_set fds;
27     int max_fd = (c1 > c2) ? c1 : c2;
28
29     printf("Da ghep cap: %d va %d\n", c1, c2);
30
31     while (1) {
32         FD_ZERO(&fds);
33         FD_SET(c1, &fds);
34         FD_SET(c2, &fds);
```

Code bài 3-02 1

```

36     if (select(max_fd + 1, &fds, NULL, NULL, NULL) < 0) {
37         break;
38     }
39
40     if (FD_ISSET(c1, &fds)) {
41         int len = recv(c1, buf1, sizeof(buf1), 0);
42         if (len <= 0) break;
43         send(c2, buf1, len, 0);
44     }
45
46     if (FD_ISSET(c2, &fds)) {
47         int len = recv(c2, buf2, sizeof(buf2), 0);
48         if (len <= 0) break;
49         send(c1, buf2, len, 0);
50     }
51 }
52
53 close(c1);
54 close(c2);
55 printf("Mot client ngat ket noi. Da dong ket noi cap %d va %d.\n", c1, c2);
56 return NULL;
57 }
58
59 int main() {
60     int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
61     if (listener == -1) {
62         perror("socket() failed");
63         return 1;
64     }
65
66     struct sockaddr_in addr = {0};
67     addr.sin_family = AF_INET;

```

Code bài 3-02 2

```

68     addr.sin_addr.s_addr = htonl(INADDR_ANY);
69     addr.sin_port = htons(8080);
70
71     if (bind(listener, (struct sockaddr *)&addr, sizeof(addr))) {
72         perror("bind() failed");
73         return 1;
74     }
75
76     if (listen(listener, 5)) {
77         perror("listen() failed");
78         return 1;
79     }
80
81     printf("Server chat 2 nguoi dang cho ket noi o cong 8080...\n");
82
83     while (1) {
84         int client = accept(listener, NULL, NULL);
85         if (client == -1) continue;
86         printf("Client moi ket noi: %d\n", client);
87
88         pthread_mutex_lock(&mutex);
89         if (waiting_client == -1) {
90             waiting_client = client;
91             char *msg = "Ban dang o trong hang doi. Cho nguoi thu 2...\n";
92             send(client, msg, strlen(msg), 0);
93         } else {
94             struct ClientPair *pair = malloc(sizeof(struct ClientPair));
95             pair->client1 = waiting_client;
96             pair->client2 = client;
97             waiting_client = -1;
98
99             char *msg = "Da tim thay nguoi ghep cap. Bat dau chat!\n";
100             send(pair->client1, msg, strlen(msg), 0);
101             send(pair->client2, msg, strlen(msg), 0);
102
103

```

Code bài 3-02 3

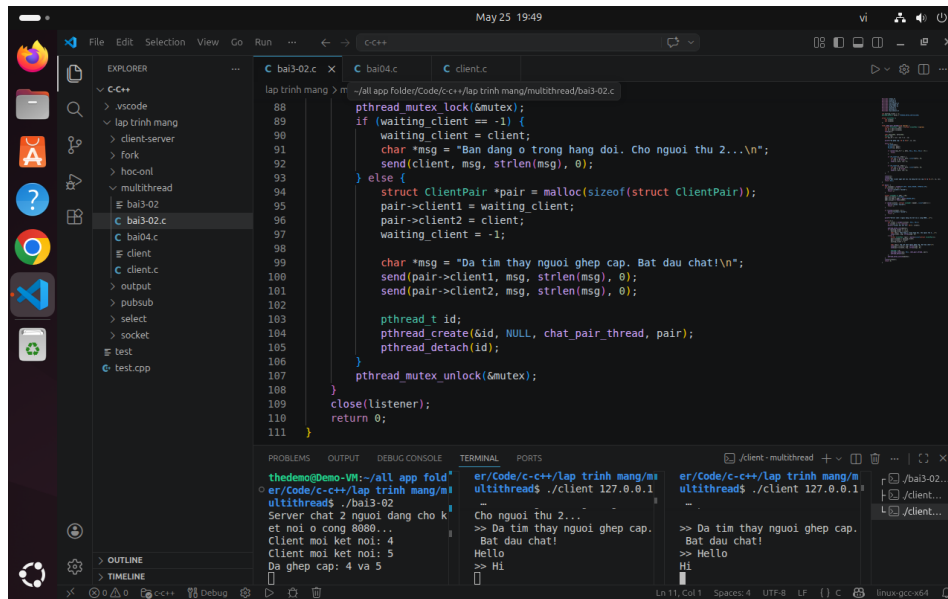
```

102
103         pthread_t id;
104         pthread_create(&id, NULL, chat_pair_thread, pair);
105         pthread_detach(id);
106     }
107     pthread_mutex_unlock(&mutex);
108 }
109 close(listener);
110 return 0;
111 }

```

Code bài 3-02 4

- **Kết quả:**



Kết quả bài 3-02

2. Bài tập 4 chat_server

```
12 int clients[MAX_CLIENTS];
13 int num_clients = 0;
14 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
15
16 void *client_thread(void *param) {
17     int client = *(int *)param;
18     char buf[256];
19
20     char *welcome = "Chao mung vao phong chat chung!\n";
21     send(client, welcome, strlen(welcome), 0);
22
23     while (1) {
24         int len = recv(client, buf, sizeof(buf) - 1, 0);
25         if (len <= 0) break;
26
27         buf[len] = '\0';
28
29         pthread_mutex_lock(&mutex);
30         for (int i = 0; i < num_clients; i++) {
31             if (clients[i] != client) {
32                 send(clients[i], buf, len, 0);
33             }
34         }
35         pthread_mutex_unlock(&mutex);
36     }
37
38     close(client);
39     pthread_mutex_lock(&mutex);
40     for (int i = 0; i < num_clients; i++) {
41         if (clients[i] == client) {
42             clients[i] = clients[num_clients - 1];
43             num_clients--;
44             break;
45         }
46     }
```

Code chat_server 1

```

52 int main() {
53     int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
54
55     struct sockaddr_in addr = {0};
56     addr.sin_family = AF_INET;
57     addr.sin_addr.s_addr = htonl(INADDR_ANY);
58     addr.sin_port = htons(9000);
59
60     bind(listener, (struct sockaddr *)&addr, sizeof(addr));
61     listen(listener, 5);
62
63     printf("Chat server (multithread) dang mo o cong 9000...\n");
64
65     while (1) {
66         int client = accept(listener, NULL, NULL);
67         if (client == -1) continue;
68
69         pthread_mutex_lock(&mutex);
70         if (num_clients < MAX_CLIENTS) {
71             clients[num_clients++] = client;
72             printf("Client moi: %d. Tong so: %d\n", client, num_clients);
73
74             pthread_t id;
75             pthread_create(&id, NULL, client_thread, &client);
76             pthread_detach(id);
77         } else {
78             char *msg = "Server da day!\n";
79             send(client, msg, strlen(msg), 0);
80             close(client);
81         }
82         pthread_mutex_unlock(&mutex);
83     }
84     return 0;
85 }

```

Code chat_server 2

Kết quả:

The screenshot shows the VS Code IDE with the chat_server code open in the editor. The terminal window at the bottom displays the output of the program. The server starts by printing "Chat server (multithread) dang mo o cong 9000...". It then receives three clients, each printing their ID and the total number of clients (1, 2, and 3 respectively). The server responds to each client with "Chao mung vao phong chat ch ung!" and "Hello". The client responses are "Bai 04", "Bai 04", and "Bai 04".

```

51
52 int main() {
53     int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
54
55     struct sockaddr_in addr = {0};
56     addr.sin_family = AF_INET;
57     addr.sin_addr.s_addr = htonl(INADDR_ANY);
58     addr.sin_port = htons(9000);
59
60     bind(listener, (struct sockaddr *)&addr, sizeof(addr));
61     listen(listener, 5);
62
63     printf("Chat server (multithread) dang mo o cong 9000...\n");
64
65     while (1) {
66         int client = accept(listener, NULL, NULL);
67         if (client == -1) continue;
68
69         pthread_mutex_lock(&mutex);
70         if (num_clients < MAX_CLIENTS) {
71             clients[num_clients++] = client;
72             printf("Client moi: %d. Tong so: %d\n", client, num_clients);
73
74             pthread_t id;
75             pthread_create(&id, NULL, client_thread, &client);
76             pthread_detach(id);
77         } else {
78             char *msg = "Server da day!\n";
79             send(client, msg, strlen(msg), 0);
80             close(client);
81         }
82         pthread_mutex_unlock(&mutex);
83     }
84     return 0;
85 }

```

```

er/Code/c-c++/lap trinh mang/m
ultithread$ ./client 127.0.0.1
-
>> Chao mung vao phong chat ch
ung!
>> Hello
>> Bai 04

er/Code/c-c++/lap trinh mang/m
ultithread$ ./client 127.0.0.1
-
>> Chao mung vao phong chat ch
ung!
>> Hello
>> Bai 04

er/Code/c-c++/lap trinh mang/m
ultithread$ ./client 127.0.0.1
-
>> Chao mung vao phong chat ch
ung!
>> Hello
>> Bai 04

```

Kết quả chat_server

3. Bài tập time_server

```

multithread > C time_server.c > time_handler(void *)
1  √ #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <arpa/inet.h>
6  #include <unistd.h>
7  #include <time.h>
8  #include <pthread.h>
9
10 √ void *time_handler(void *arg) {
11     int client_fd = *(int *)arg;
12     free(arg);
13     char buf[256];
14
15     while (1) {
16         int ret = recv(client_fd, buf, sizeof(buf) - 1, 0);
17         if (ret <= 0) break;
18         buf[ret] = 0;
19         while(ret > 0 && (buf[ret-1] == '\n' || buf[ret-1] == '\r')) buf[--ret] = 0;
20
21         char cmd[20], format[50], output[50];
22         if (sscanf(buf, "%s %s", cmd, format) == 2 && strcmp(cmd, "GET_TIME") == 0) {
23             time_t rawtime; struct tm *info;
24             time(&rawtime); info = localtime(&rawtime);
25
26             if (strcmp(format, "dd/mm/yyyy") == 0) strftime(output, 50, "%d/%m/%Y\n", info);
27             else if (strcmp(format, "dd/mm/yy") == 0) strftime(output, 50, "%d/%m/%y\n", info);
28             else if (strcmp(format, "mm/dd/yyyy") == 0) strftime(output, 50, "%m/%d/%Y\n", info);
29             else if (strcmp(format, "mm/dd/yy") == 0) strftime(output, 50, "%m/%d/%y\n", info);
30             else strcpy(output, "ERROR: Invalid format\n");
31
32             send(client_fd, output, strlen(output), 0);
33         } else {
34             char *err = "ERROR: Use GET_TIME [format]\n";

```

Code time_server 1

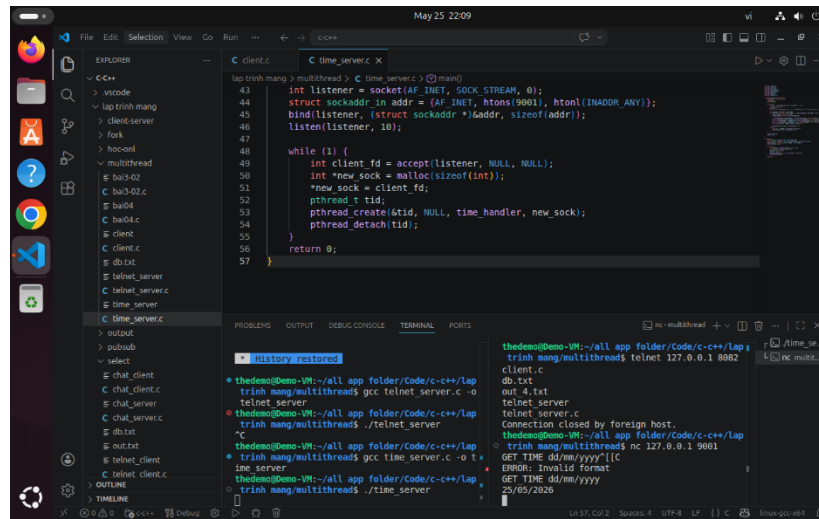
```

multithread > C time_server.c > time_handler(void *)
10 void *time_handler(void *arg) {
15     while (1) {
33     } else {
35         send(client_fd, err, strlen(err), 0);
36     }
37 }
38 close(client_fd);
39 return NULL;
40 }
41
42 int main() {
43     int listener = socket(AF_INET, SOCK_STREAM, 0);
44     struct sockaddr_in addr = {AF_INET, htons(9001), htonl(INADDR_ANY)};
45     bind(listener, (struct sockaddr *)&addr, sizeof(addr));
46     listen(listener, 10);
47
48     while (1) {
49         int client_fd = accept(listener, NULL, NULL);
50         int *new_sock = malloc(sizeof(int));
51         *new_sock = client_fd;
52         pthread_t tid;
53         pthread_create(&tid, NULL, time_handler, new_sock);
54         pthread_detach(tid);
55     }
56     return 0;
57 }

```

Code time_server 2

- Kết quả:



The screenshot shows a VS Code editor with a C++ file named `time_server.c`. The code implements a multi-threaded telnet server. The `main` function sets up a listener socket on `127.0.0.1 8002` and enters a loop where it accepts connections. Each connection is handled by a new thread (`pthread_create`) that reads data from the client and writes it to `out.txt`. The server is compiled and run in a terminal window. The output shows the server successfully accepting connections and processing data from a client.

```
lap trinh mang > multithread > C time_server.c > @ main()
43 int listener = socket(AF_INET, SOCK_STREAM, 0);
44 struct sockaddr_in addr = {AF_INET, htons(8002), htonl(INADDR_ANY)};
45 bind(listener, (struct sockaddr *)&addr, sizeof(addr));
46 listen(listener, 10);
47
48 while (1) {
49     int client_fd = accept(listener, NULL, NULL);
50     int *new_sock = malloc(sizeof(int));
51     *new_sock = client_fd;
52     pthread_t tid;
53     pthread_create(&tid, NULL, time_handler, new_sock);
54     pthread_detach(tid);
55 }
56 return 0;
57 }
```

Terminal Output:

```
thetemo@Demo-VN:~/all app folder/Code/c-c++/lap trinh mang/multithread$ gcc time_server.c -o time_server
thetemo@Demo-VN:~/all app folder/Code/c-c++/lap trinh mang/multithread$ ./time_server
thetemo@Demo-VN:~/all app folder/Code/c-c++/lap trinh mang/multithread$ nc 127.0.0.1 8002
GET TIME dd/mm/yyyy
25/05/2026
Connection closed by foreign host.
```

4. Bài tập telnet_server

```
multithread > C telnet_server.c > telnet_handler(void *)
9  int check_login(char *user, char *pass) {
10  FILE *f = fopen("db.txt", "r");
11  if (!f) return 0;
12  char u[50], p[50];
13  while (fscanf(f, "%s %s", u, p) != EOF) {
14      if (strcmp(user, u) == 0 && strcmp(pass, p) == 0) { fclose(f); return 1; }
15  }
16  fclose(f); return 0;
17  }
18
19  void *telnet_handler(void *arg) {
20  int client_fd = *(int *)arg;
21  free(arg);
22  int authenticated = 0;
23  char buf[256];
24
25  send(client_fd, "User Pass: ", 11, 0);
26
27  while (1) {
28      int ret = recv(client_fd, buf, sizeof(buf) - 1, 0);
29      if (ret <= 0) break;
30      buf[ret] = 0;
31      while(ret > 0 && (buf[ret-1] == '\n' || buf[ret-1] == '\r')) buf[--ret] = 0;
32
33      if (!authenticated) {
34          char user[50], pass[50];
35          if (sscanf(buf, "%s %s", user, pass) == 2 && check_login(user, pass)) {
36              authenticated = 1;
37              send(client_fd, "Login success. Enter command: ", 30, 0);
38          } else {
39              send(client_fd, "Login failed. Try again: ", 25, 0);
40          }
41      } else {
42          char cmd[512], out_file[50];
```

Code telnet_server 1

```
multithread > C telnet_server.c > telnet_handler(void *)
19  void *telnet_handler(void *arg) {
27  while (1) {
41  } else {
46
47      FILE *f = fopen(out_file, "r");
48      if (f) {
49          char result[2048];
50          size_t bytes = fread(result, 1, sizeof(result)-1, f);
51          result[bytes] = 0;
52          fclose(f);
53          send(client_fd, bytes > 0 ? result : "Command executed.\n", bytes > 0 ? bytes : 18,
54              0);
55          remove(out_file);
56      }
57  }
58  close(client_fd);
59  return NULL;
60  }
61
62  int main() {
63  int listener = socket(AF_INET, SOCK_STREAM, 0);
64  struct sockaddr_in addr = {AF_INET, htons(8082), htonl(INADDR_ANY)};
65  bind(listener, (struct sockaddr *)&addr, sizeof(addr));
66  listen(listener, 10);
67
68  while (1) {
69      int client_fd = accept(listener, NULL, NULL);
70      int *new_sock = malloc(sizeof(int));
71      *new_sock = client_fd;
72      pthread_t tid;
73      pthread_create(&tid, NULL, telnet_handler, new_sock);
74      pthread_detach(tid);
75  }
76  return 0;
```

Code telnet_server 2

Kết quả:

```
May 25 22:07
File Edit Selection View Go Run ... C/C++
EXPLORER
  C/C++
  > .vscode
  > lap_trinh_mang
  > client_server
  > fork
  > hoc-onl
  > multithread
  > bai3-02
  > C bai3-02.c
  > bai04
  > C bai04.c
  > client
  > C client.c
  > db.txt
  > E telnet_server
  > C telnet_server.c
  > output
  > pubsub
  > select
  > E chat_client
  > C chat_client.c
  > E chat_server
  > C chat_server.c
  > E db.txt
  > E output
  > E telnet_client
  > C telnet_client.c
  > E telnet_server
  > C telnet_server.c
  > OUTLINE
  > TIMELINE
  > C/C++
  > Debug
  > Run
  > C/C++
  > C telnet_server.c
  > db.txt
  > C client.c
  > C telnet_server.c
  > main()
  > void *telnet_handler(void *arg) {
  > }
  > int main() {
  > int listener = socket(AF_INET, SOCK_STREAM, 0);
  > struct sockaddr_in addr = {AF_INET, htons(8082), htonl(INADDR_ANY)};
  > bind(listener, (struct sockaddr *)&addr, sizeof(addr));
  > listen(listener, 10);
  > while (1) {
  > int client_fd = accept(listener, NULL, NULL);
  > int *new_sock = malloc(sizeof(int));
  > *new_sock = client_fd;
  > pthread_t tid;
  > pthread_create(&tid, NULL, telnet_handler, new_sock);
  > pthread_detach(tid);
  > }
  > return 0;
  > }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
thetdemo@Demo-VN:~/all_app_folder/Code/c-c++/lap_trinh_mang/multithread$ ./bai04
Chat server (multithread) dang mo o cong 9000..
Client moi: 4. Tong so: 1
Client moi: 5. Tong so: 2
+ History restored
thetdemo@Demo-VN:~/all_app_folder/Code/c-c++/lap_trinh_mang/multithread$ gcc telnet_server.c -o telnet_server
thetdemo@Demo-VN:~/all_app_folder/Code/c-c++/lap_trinh_mang/multithread$ ./telnet_server
sh: 1: Hello: not found
ls
bai04
bai04.c
bai3-02
bai3-02.c
client
client.c
db.txt
out 4.txt
telnet_server
telnet_server.c
```